

Non-Specialist Governance, Not Non-Specialist Authorship: What the Coordination Knowledge Substrate Pattern Makes Available to Whom

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 28 April 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the distinction between non-specialist *governance* (which the source paper commits to as an architectural property) and non-specialist *authorship* (which the source paper does not), so that downstream work invoking CKS's accessibility claim can do so without conflating the two.

Abstract

Claim 5 of the CKS pattern includes an architectural commitment to non-specialist governance: a non-specialist must be able to inspect, modify, and override substrate content and orchestration rules in tools they already use. The commitment is widely cited as evidence that CKS systems are accessible. It is also widely conflated with a stronger and different claim — that non-specialists can perform all the work CKS systems require, including substrate schema design, orchestration rule authoring, and cell construction. The source paper distinguishes the two in §7.4, but the distinction is made as one positioning move inside Claim 5's defense rather than as a consolidated architectural statement, and the conflation is the most common misreading of CKS's accessibility claim. This note formalizes the distinction. Non-specialist governance is the architectural commitment that the rights named by *human-governed* (Li, 24 April 2026) are exercisable by humans without specialist expertise. Non-specialist authorship is a different commitment the architecture neither requires nor forbids; the source paper supports it as a labor mode, the proof-of-concept demonstrates it in a regulated-industry setting, but the design pattern does not commit to it as a property. The note states the definition, identifies the kinds of authorship work CKS systems contain and which roles each requires, traces the architectural property to the conjunction of three commitments that produce it, and provides an operational test.

1. Why the distinction needs to be named

The CKS pattern's commitment to non-specialist governance is the architectural anchor of its accessibility claim — that the pattern can be adopted bottom-up, in commodity tools, by humans who are not specialists in AI, in software engineering, or in the domain the substrate covers. The commitment is what allows CKS to address the bottom-up adoption pattern §7 of the source paper documents: structured AI-mediated work happening inside ordinary tools, governed by ordinary users, without waiting for specialist infrastructure to arrive.

The accessibility claim is easily misread. The most common misreading collapses governance and authorship: if non-specialists can govern a CKS substrate, the misreading goes, then non-specialists must be able to do all the work CKS systems require — design the schema, author the orchestration rules, build the cells, integrate the substrate with external systems. The source paper rules out this misreading explicitly in §7.4, but it does so as one positioning move in the body of Claim 5's defense, not as a consolidated architectural statement. This note provides that statement, in the form the human-governed note (Li, 24 April 2026) used for "human-governed": as a property of the architecture, exercisable under specific conditions, and distinguishable from neighboring commitments it is commonly conflated with.

2. The definition: non-specialist governance, precisely

A CKS substrate is **non-specialist-governed** if and only if the rights formalized in the human-governed note — to inspect, modify, and override substrate content and orchestration rules at any time — are exercisable by humans who satisfy two conditions:

1. They have no specialist expertise in artificial intelligence, in software engineering, or in the architecture of the substrate itself.
2. They have access only to the host environment's commodity affordances — the direct human read/write access named by Requirement 2 of tool-agnosticism (§7.1) — without specialized governance tooling layered above the host.

The commitment is *architectural*: the rights must be exercisable under those two conditions as a property of the system's design, not as a procedural promise that depends on a particular vendor, deployment, or workflow. A system in which the inspect, modify, or override rights require specialist tooling, specialist intermediation, or specialist knowledge to exercise is not non-specialist-governed in the CKS sense, regardless of how often non-specialists are in fact authorized to operate it.

What the commitment requires of the system is minimal and inherits directly from the human-governed and tool-agnosticism commitments: read access to substrate content (Requirement 2 of tool-agnosticism), write access to substrate content (also Requirement 2), and the architectural property that no LLM, vendor, or runtime middleware can in principle prevent the read or write. What the commitment does *not* require of the governor is the substantive content of §3.

The commitment is bounded in three ways: it does not claim non-specialists *will* exercise governance, only that they *can*; it does not claim governance is *easier* than specialist work, only *exercisable* without specialist expertise; and it does not *exclude* specialists from governance, only refuse to require them.

3. What the architecture does not assume of governors: authorship by role

To make the asymmetry between governance and authorship architectural rather than rhetorical, this section identifies the kinds of authorship work CKS systems contain. The enumeration is not normative. A single human may hold several roles; a small deployment may have one human hold all four; a larger deployment may distribute them across specialists with a much larger population of governors. The list states what kinds of authorship the architecture distinguishes from governance, not how a deployment must organize them.

(a) Substrate schema authorship. Designing what fields, types, relationships, and constraints the substrate holds — what entities the schema represents, what attributes each carries, what relationships connect them, and which conflict-preservation semantics apply where. Requires understanding of the coordination work

the substrate represents and of basic schema design. The result is what governors govern; designing it is not an exercise of governance.

(b) Orchestration rule authorship. Writing the rules that govern cell behavior — who can modify what, how preserved conflicts are handled, what conditions trigger which response, what the cell is authorized to do on the substrate's behalf under the AI-as-substrate-mediator commitment. Requires understanding of how cells operate over substrate content. Once authored, the rules become substrate content and are then governable by non-specialists under the inspect, modify, and override rights; authoring them in the first place is not.

(c) Cell construction. Building the cells that execute over substrate content — selecting and configuring the LLM components that mediate over the substrate (per §4.2), integrating with the host environment, wiring orchestration rules to cell behavior. Requires understanding of LLM mediation, of orchestration, and of the substrate-cell boundary. This is the most architecturally specialist of the four roles.

(d) Initial substrate content authorship. Drafting the initial coordination state — the entities, relationships, decisions, and rationale the substrate begins with — and adding to it as new coordination work appears. Requires domain knowledge but not architectural expertise. This category sits closer to governance than (a), (b), or (c): drafting an initial entity or recording a new decision is an exercise of the modify right against an empty or existing substrate, not a design move on the substrate itself. Source-paper §7.4 treats authorship in this sense as POC-demonstrated rather than architecturally required, and the architecture's accessibility property bears most directly on this category, even though it does not commit to it as a property.

Roles (a)–(c) require architectural expertise the design pattern does not assume of governors; role (d) requires domain knowledge it also does not assume. What non-specialist governance names is not whether non-specialists can perform any of (a)–(d), but that the rights to inspect, modify, and override substrate content and orchestration rules — once any exist — are exercisable without specialist expertise.

4. Why the property is architectural, not organizational

The asymmetry between governance accessibility and authorship requirements could be a property of how a deployment is organized — schema designers and rule authors hired in, ordinary users brought in to govern. It is not. Three architectural commitments together produce the accessibility property as a property of the design pattern, not of any particular deployment.

Tool-agnosticism's three minimal requirements (§7.1). Because the substrate's host environment must support persistent structured state, direct human read/write access, and LLM access to substrate content — and nothing more — governance is realizable in commodity tools that ordinary knowledge workers already use. If the architecture required specialized hosting, governance would become specialist work by virtue of requiring specialist tools to perform.

The human-governed commitment as authority-not-labor. As formalized in the human-governed note, governance is exercised at two moments — orchestration rule authoring and direct override — neither of which scales with substrate size and neither of which requires per-element review by humans. If governance required per-element review, the cost would force specialization, because only specialists could afford the labor.

Linear-cost scaling (Claim 4, §6). Because governance cost does not grow with substrate size — the cost lives in rule variety and intervention frequency, both bounded by the system's design rather than by how much content the substrate holds — accessible governance remains accessible as the substrate grows. If governance cost grew superlinearly, only specialists could operate at scale, and the bottom-up adoption pattern the architecture preserves would foreclose itself by success.

The conjunction is what produces the architectural property. None of the three alone is sufficient: tool-agnosticism without authority-not-labor would still require non-specialists to perform per-element review in commodity tools; authority-not-labor without tool-agnosticism would require specialist tools to exercise authority; either of those without linear-cost scaling would render the property fragile under substrate growth. Together, the three make non-specialist governance hold by construction in any system that satisfies them — a property of the design pattern itself, not of any deployment configured to exhibit it.

5. Implications and three misreadings

Four downstream consequences follow from the architectural property.

Bottom-up adoption. Teams without specialist staff can begin using a CKS substrate and exercise governance over it from the start, growing the substrate as work accumulates without first acquiring AI or software-engineering expertise.

Distributed authority. Multiple humans across an organization can govern the same substrate without each holding the same authorship expertise. Specialists author rules, schemas, and cells; non-specialists govern the substrate that results.

Accessibility under change. As tools, models, and deployment environments evolve — which tool-agnosticism explicitly accommodates — governance remains exercisable in the same commodity-tool sense. Governors do not need to retrain on a new platform each time the underlying implementation changes, because the rights they exercise depend on the host's three minimal requirements rather than on any platform-specific affordance.

Audit and oversight. Roles whose function is oversight — auditors, compliance staff, leadership — can exercise governance rights to verify the substrate's state without becoming specialists in the architecture. Path retraceability (in the sense §3.1 imports from Rajabi and Kafaie) is exercisable by anyone with the inspect right.

Three misreadings of the property are equally important to head off.

Misreading 1 — non-specialist authorship as an architectural commitment. Treating the accessibility claim as if it extended to schema design, rule authoring, or cell construction. The architecture does not commit to this. A deployment that pursues it as a goal — by hiring non-specialists who happen to author well, by adding LLM-driven schema-design assistance, by training non-specialists in rule authoring — may succeed, but the success is a deployment achievement, not a property of the pattern. Source-paper §7.4 is explicit: non-specialist authorship is POC-demonstrated, not architecturally claimed.

Misreading 2 — specialist-only governance. Treating governance as if it required the same expertise as authorship. This misreading turns the architecture into a specialist tool and forecloses the bottom-up adoption property the architecture preserves. If governance is reserved for those who could also author rules, schemas, or cells, the substrate becomes unreachable to the population the accessibility commitment was

designed to serve.

Misreading 3 — conflating governance scope with substrate scope. Treating governance as if it covered every decision about the substrate, including decisions about what cells to build, what schemas to design, or how to integrate with external systems. Governance covers the three rights (inspect, modify, override) over substrate content and orchestration rules; it does not cover decisions about authorship. Those decisions are made by humans in authorship roles and may themselves be governed by other architectures, but they are not exercises of CKS governance.

6. Operational test

A system implements the non-specialist governance commitment if and only if all of the following are true at all times during the substrate's existence:

1. Governance rights (inspect, modify, override over substrate content and orchestration rules) are exercisable in the host environment using only the affordances Requirement 2 of tool-agnosticism specifies, without specialized governance tooling layered above the host.
2. Exercising any of the three rights does not require knowledge of how the substrate's schema was designed, how orchestration rules were authored, or how cells were constructed. A governor with the inspect right can read substrate content without knowing which cell produced it; a governor with the modify right can edit a value without knowing what schema-design rationale motivated the field; a governor with the override right can change an LLM-produced value or a rule's effect without knowing how the rule was implemented.
3. Governance cost does not grow with substrate size. A governor's cost to read or modify any authorized content is proportional to what they read or modify, not to total substrate size, inheriting the linear-cost commitment of Claim 4.
4. The architecture does not require governance and authorship roles to be held by the same humans. Deployments may co-locate them; the architecture does not.

A system that fails any of (1)–(4) may be CKS-coherent in some other sense — a specialist-governed deployment in a regulated context where governors are specialists by deployment choice, for example — but does not preserve the non-specialist governance commitment as an architectural property.

7. Conclusion

Non-specialist governance is what CKS commits to as an architectural property: the rights to inspect, modify, and override substrate content and orchestration rules are exercisable by humans without specialist expertise, in commodity tools, at any time, with cost that does not grow with substrate size. Non-specialist authorship is a different commitment the architecture neither requires nor forbids. The source paper supports it as a labor mode and the proof-of-concept demonstrates it in one regulated-industry setting; the design pattern does not claim it as a property.

The distinction has practical consequences at design time. Implementations that conflate non-specialist governance with non-specialist authorship overpromise what teams without specialist staff can do, and frequently reach for tooling additions to close the gap — LLM-driven schema designers, rule synthesizers, automatic cell builders. Those additions may be useful. They are not what non-specialist governance names, and treating them as required undermines the architectural accessibility claim by making it depend on

tooling the architecture does not commit to. Implementations that conflate governance with specialist work, in the opposite direction, produce systems that are CKS-coherent but unnecessarily inaccessible — foreclosing the bottom-up adoption property the architecture preserves.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use "non-specialist governance" in the sense formalized here, and should treat "non-specialist authorship" as a separate commitment that may or may not be pursued in any given deployment. The accessibility claim CKS makes is precise: governance is exercisable by non-specialists by construction; authorship is not assumed of governors.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Companion note

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern*. 24 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Non-Specialist Governance, Not Non-Specialist Authorship: What the Coordination Knowledge Substrate Pattern Makes Available to Whom*. 28 April 2026. ORCID: 0009-0004-8065-3235.